

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3.6.2

UPGRADE TO 2.0

 Copy as Markdown

Upgrade to 2.0

Changes to authentication

The base auth implementation has been removed from Serverpod core and moved into the `serverpod_auth` package. If you are not using authentication at all this change does not impact you. If you are using the auth module already the transition is simple.

The default authentication handler will now throw an `UnimplementedError`. It is now required to supply the authentication handler to the Serverpod object, in your `server.dart` file make the following change:

```
import 'package:serverpod_auth_server/serverpod_auth_server.dart' as auth;

void run(List<String> args) async {
  var pod = Serverpod(
    args,
    Protocol(),
    Endpoints(),
    authenticationHandler: auth.authenticationHandler, // Add this line
  );

  ...
}
```

Advanced integrations

The methods `signInUser` and `signOutUser` now takes the session object as a param and is no longer available on the session object. Instead import the class `UserAuthentication` from the auth module to access these static methods.

```
UserAuthentication.signInUser(session, userId, 'provider');

UserAuthentication.signOutUser(session);
```



The table `serverpod_auth_key` has been removed from Serverpod core but is available in the `serverpod_auth` module instead. This means that if you wrote a custom integration before without using the `serverpod_auth` module you have to take care of managing your token implementation.

Adding the definition of the `serverpod_auth_key` table to your project is the simplest way to do a seamless migration.

The table was defined in the following way:

```
### Provides a method of access for a user to authenticate with the server.
class: AuthKey
table: serverpod_auth_key
fields:
  ### The id of the user to provide access to.
  userId: int

  ### The hashed version of the key.
  hash: String

  ### The key sent to the server to authenticate.
  key: String?, !persist

  ### The scopes this key provides access to.
  scopeNames: List<String>

  ### The method of signing in this key was generated through. This can be email
  ### or different social logins.
  method: String
indexes:
  serverpod_auth_key_userId_idx:
    fields: userId
```

You are then responsible for creating/removing entries in this table, the old `signInUser` and `signOutUser` that used to provide this functionality can be found [here](#).

Changes to the Session Object

Removed deprecated fields

With Serverpod 2.0, we have removed the deprecated legacy database layer from the `Session` object. The `Session` object now incorporates the new database layer, accessed via the `dbNext` field in Serverpod 1.2, under the `db` field.

```
session.dbNext.find(...);
```

becomes

```
session.db.find(...);
```

Authenticated user information retrieval

In Serverpod 2.0, we have removed the getters `scopes` and `authenticatedUser` from session. This information is now retrievable through the `authenticated` getter as fields of the returned object.

Replace this:

```
int? userId = await session.auth.authenticatedUser;  
  
Set<Scopes>? scopes = await session.scopes;
```

With this:

```
final authenticated = await session.authenticated;  
  
//Read authenticated userId  
int? userId = authenticated?.userId;  
  
//Read scopes  
Set<Scopes>? scopes = authenticated?.scopes;
```

If the `authenticated` property is set on the session it effectively means there is an authenticated user making the request.

Authentication helpers

The field `auth` has been removed and the methods `signInUser` and `signOutUser` have been moved to the `serverpod_auth` module.

Changes to database queries

Removed unsafeQueryMappedResults(...)

The `unsafeQueryMappedResults(...)` method has been removed. A similar result can now instead be formatted from the `unsafeQuery(...)` result by calling the `toColumnMap()` method for each row of the result. `toColumnMap` returns a map containing the query alias for the column as key and the row-column value as value.

Given a query that performs a join like this:

```
SELECT  
  "company"."id" AS "company.id",
```

```

"company"."name" AS "company.name",
"company"."townId" AS "company.townId",
"company_town_town"."id" AS "company_town_town.id",
"company_town_town"."name" AS "company_town_town.name",
"company_town_town"."mayorId" AS "company_town_town.mayorId"
FROM
  "company"
LEFT JOIN
  "town" AS "company_town_town" ON "company"."townId" = "company_town_town"."id"
ORDER BY
  "company"."name"

```

The return type from `unsafeQueryMappedResults(...)` in 1.2 was:

```

[
  {
    "company": {
      "company.id": 40,
      "company.name": "Apple",
      "company.townId": 64
    },
    "town": {
      "company_town_town.id": 64,
      "company_town_town.name": "San Francisco",
      "company_town_town.mayorId": null
    }
  },
  {
    "company": {
      "company.id": 39,
      "company.name": "Serverpod",
      "company.townId": 63
    },
    "town": {
      "company_town_town.id": 63,
      "company_town_town.name": "Stockholm",
      "company_town_town.mayorId": null
    }
  }
]

```

And if `result.map((row) => row.toColumnMap())` is used to format the result from `unsafeQuery(...)` in 2.0, the following result is obtained:

```

[
  {
    "company.id": 38,
    "company.name": "Apple",
    "company.townId": 62,
    "company_town_town.id": 62,
    "company_town_town.name": "San Francisco",
    "company_town_town.mayorId": null
  },

```

```
{
  "company.id": 37,
  "company.name": "Serverpod",
  "company.townId": 61,
  "company_town_town.id": 61,
  "company_town_town.name": "Stockholm",
  "company_town_town.mayorId": null
}
```

or for a simple query without aliases:

```
SELECT
  "id",
  "name",
  "townId"
FROM
  "company"
ORDER BY
  "name"
```

the return type from `unsafeQueryMappedResults(...)` in 1.2 was:

```
[
  {
    "company": {
      "id": 54,
      "name": "Apple",
      "townId": 86
    }
  },
  {
    "company": {
      "id": 53,
      "name": "Serverpod",
      "townId": 85
    }
  }
]
```

and if `result.map((row) => row.toColumnMap())` is used to format the result from `unsafeQuery(...)` in 2.0, the following result is obtained:

```
[
  {
    "id": 54,
    "name": "Apple",
    "townId": 86
  },
  {
```

```

    "id": 53,
    "name": "Serverpod",
    "townId": 85
  }
]

```

Update return type for delete operations

The return type for all delete operations has been changed from the `id` of the deleted rows to the actual deleted rows. This makes the return type for the delete operations consistent with the return type of the other database operations. It also dramatically simplifies retrieving and removing rows in concurrent environments.

Return type before the change:

```

int companyId = await Company.db.deleteRow(session, company);
List<int> companyIds = await Company.db.delete(session, [company]);
List<int> companyIds = await Company.db.deleteWhere(session, where: (t) =>
t.name.like('%Ltd'));

```

Return types after the change:

```

Company company = await Company.db.deleteRow(session, company);
List<Company> companies = await Company.db.delete(session, [company]);
List<Company> companies = await Company.db.deleteWhere(session, where: (t) =>
t.name.like('%Ltd'));

```

Changes to database tables

Integer representation changed to bigint

Integer representation in the database has changed from `int` to `bigint`. From now on, models with `int` fields will generate database migrations where that field is defined as a `bigint` type in the database.

This change also applies to the `id` field of models where `bigserial` is now used to generate the id.

The change is compatible with existing databases. Existing migrations therefore, won't be changed by the Serverpod migration system. No manual modification to the database is required if this data representation is not essential for the application. However, all new migrations will be created with the new representation.

Why is this change made?

The change was made to ensure that Dart and the database representation of integers is consistent. Dart uses 64-bit integers, and the `int` type in Dart is a 64-bit integer. The `int` type in PostgreSQL is a 32-bit integer.

This means that the `int` type in Dart can represent larger numbers than the `int` type in PostgreSQL. By using `bigint` in PostgreSQL, the integer representation is consistent between Dart and the database.

In terms of performance, there are usually no significant drawbacks with using `bigint` instead of `int`. In most cases a good index strategy will be more important than the integer representation. Here is a guide that benchmarks the performance of `int` and `bigint` in PostgreSQL: [Use BIGINT in Postgres](#)

Ensuring new databases are created with the new representation

Since existing migrations won't be changed, databases that are created with these will still use `int` to represent integers.

To ensure new databases are created with the new representation, the latest migration should be generated using Serverpod 2.0. It is enough to have an empty migration to ensure new databases use the new representation.

A new empty migration can be created by running the following command in the terminal:

```
$ serverpod create-migration --force
```

Migration of existing tables

The migration of existing tables to use the new representation will vary depending on the database content. Utilizing the wrong migration strategy might cause downtime for your application. That is the reason Serverpod does not automatically migrate existing tables.

Small tables

A simple way to migrate for small tables is to execute the following sql query to the database:

```
ALTER SEQUENCE "my_table_id_seq" AS bigint;  
ALTER TABLE "my_table" ALTER "id" TYPE bigint;  
ALTER TABLE "my_table" ALTER "myNumber" TYPE bigint;
```

The first two lines modify the id sequence for a table named `"my_table"` to use `bigint` instead of `int`. The last line modifies a column of the same table to use `bigint`. The drawback of this approach is that it locks the table during the migration. Therefore, this strategy is not recommended for large tables.

Large tables

Migrating large tables without application downtime is a more complex operation, and the approach will vary depending on the data structure. Below are some gathered resources on the subject.

- [Zemata - Column migration from INT to BIGINT](#)
- [AM^2 - Changing a column from int to bigint, without downtime](#)
- [Crunch data - The integer at the End of the Universe](#)

Changes in the authentication module

Unsecure random disabled by default

The authentication module's default value for allowing unsecure random number generation is now `false`. An exception will be thrown when trying to hash a password if no secure random number generator is available. To preserve the old behavior and enable unsecure random number generation, set the `allowUnsecureRandom` property in the `AuthConfig` to `true`.

```
auth.AuthConfig.set(auth.AuthConfig(  
    allowUnsecureRandom: true,  
));
```

Updates to Serialization in Serverpod 2.0

General Changes to Model Serialization

Serverpod 2.0 significantly streamlines the model serialization process. In earlier versions, the `fromJson` factory constructors needed a `serializationManager` parameter to handle object deserialization. This parameter has now been removed, enhancing simplicity and usability.

Before change

```
final Map<String, dynamic> json = classInstance.toJson();  
final SerializationManager serializationManager = Protocol();  
final ClassName test = ClassName.fromJson(json, serializationManager);
```

After change

```
final Map<String, dynamic> json = classInstance.toJson();  
final ClassName test = ClassName.fromJson(json);
```

Enhancements for Custom Serialization

The removal of the `serializationManager` parameter in Serverpod 2.0 simplifies the serialization process not only for general models but also significantly enhances custom serialization workflows. For custom classes that previously utilized unique serialization logic with the `serializationManager`, adjustments may be necessary.

Previous Implementation

In the previous versions, models required the `serializationManager` to be passed explicitly, as shown in the following code snippet:

```
factory ClassName.fromJson(  
    Map<String, dynamic> json,  
    SerializationManager serializationManager,  
) {  
    return ClassName(  
        json['name'],  
    );  
}
```

Updated Implementation

With the release of Serverpod 2.0, the `fromJson` constructor has been simplified and the `serializationManager` has been removed:

```
factory ClassName.fromJson(  
    Map<String, dynamic> json,  
) {  
    return ClassName(  
        json['name'],  
    );  
}
```

Deprecation Notice for `SerializableEntity`

The `SerializableEntity` class is deprecated and will be removed in version 3. Please implement the `SerializableModel` interface instead for creating serializable models.

Migration Guide

To migrate your code from `SerializableEntity` to `SerializableModel`, replace `extends SerializableEntity` with `implements SerializableModel` in your model classes.

Example

Before:

```
class CustomClass extends SerializableEntity {  
    // Your code here  
}
```

After:

```
class CustomClass implements SerializableModel {  
    // Your code here  
}
```